

Decision Tree on Framingham Heart Study Dataset

1. About the Framingham Heart Study

The **Framingham Heart Study** is a large, long-term medical research project in the USA, designed to identify the major factors contributing to cardiovascular disease.

The study collects extensive health information from thousands of participants, including:

- Age and gender
- Smoking habits and lifestyle factors
- Blood pressure measurements (sysBP, diaBP)
- Cholesterol levels
- Body Mass Index (BMI)
- Diabetes and hypertension status
- Other clinical indicators related to heart health

This dataset has become one of the most important and widely used resources for building predictive models in cardiovascular research.

2. Overview of the Project

The main goal of this project is to build a Decision Tree model to predict cardiovascular risk using the Framingham Heart Study dataset. Multiple machine learning techniques were applied to improve performance, tested in several stages.

3. Data Cleaning & Handling Missing Values

Several strategies were applied to handle missing data:

- Removing rows with excessive missing values
- Filling missing values using **mean** or **median**
- Comparing multiple preprocessing versions to find the most effective approach

Proper handling of missing values was crucial to ensure stable model performance.

4. Class Imbalance Handling

The dataset is highly imbalanced (few positive heart disease cases). Multiple methods were tested to balance the data. Each method created a different model version, and the results were compared to choose the most effective.

5. Hyperparameter Tuning

The Decision Tree model was optimized using GridSearchCV, testing different combinations of parameters:

- Class weight options
- Maximum tree depth (max_depth)
- Minimum samples per leaf (min_samples_leaf) and per split (min_samples_split)
- Cost-complexity pruning parameter (ccp_alpha)

This helped select the best-performing model.

6. Decision Tree Optimization (Pruning)

To prevent overfitting and improve generalization, pruning techniques were applied:

- Limiting max_depth
- Adjusting min_samples_split and min_samples_leaf
- Testing ccp_alpha for cost-complexity pruning

Different combinations were tested using grid search to find the optimal configuration.

7. Feature Engineering

New features were created to enhance model predictive power, such as:

- **Pulse Pressure** = sysBP – diaBP
- Interaction features (e.g., age × BMI, age × smoking status)

These engineered features helped the model better capture medical relationships.

8. Multiple Model Versions

Different versions of the Decision Tree were trained and evaluated:

- Baseline model (no preprocessing)
- Cleaned dataset version
- Feature-engineered version

- Balanced dataset version
- Fully optimized version with Grid Search

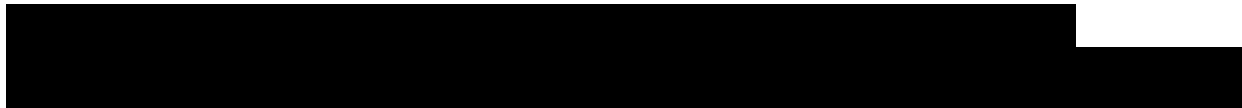
This iterative approach allowed identification of the most impactful preprocessing and tuning steps.

9. Adding New Patients (Synthetic Data Generation)

To simulate new patient records, we used the CTGAN library, which can train a generative model to produce synthetic data resembling the real dataset.

CTGAN allows the creation of privacy-preserving, realistic patient data, useful for additional testing or model validation.

10. Saving and Exporting the Model (.pkl)



11. Final Model Performance

The fully optimized Decision Tree achieved significantly better results compared to the baseline:

- **Accuracy** improved
- **Precision** improved
- **Recall** improved

This demonstrates the effectiveness of preprocessing, feature engineering, balancing, and hyperparameter optimization.

Dataset Source

- **GitHub:** [framingham-heart-study-dataset/framingham.csv](https://github.com/robertblackwell/framingham-heart-study-dataset)
- **Kaggle:** [Framingham Heart Study Dataset](https://www.kaggle.com/datasets/framingham/framingham-heart-study-dataset)

All Python code versions in this project use the real Framingham.csv data from GitHub.

ABOUT WHAT HAPPENED IN EACH VERSION

First Version (framinghamV0.py)

After downloading framingham.csv from GitHub, we ran this first version on the raw dataset. This version used a basic Decision Tree Classifier without any preprocessing, feature engineering, or class balancing.

Output of run in this image:

```
framinghamV0.py X
framinghamV0.py > ...
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score, balanced_accuracy_score, classification_report, c
5
6
7 # Load dataset
8 df = pd.read_csv("../data/framingham.csv")
9
10 # Features and target
11 target_col = "TenYearCHD"
12 X = df.drop(target_col, axis=1)
```

```
[[112  82]
 [ 85 108]]
PS C:\Users\10User\Desktop\projectML> python -u "c:\Users\10User\Desktop\projectML\framinghamV0.py"

Accuracy: 0.7382075471698113
Balanced Accuracy: 0.5159618145759602

Classification Report:
      precision    recall  f1-score   support

0         0.85        0.84        0.84       1079
1         0.18        0.20        0.19        193

accuracy          0.74       1272
macro avg         0.51        0.52        0.51       1272
```

We evaluated the model using `sklearn.metrics` and found that the accuracy is around 73/100. Although this may seem good at first, the model does not perform well for detecting people at risk of heart disease.

The main problem is that the model has low recall, precision, and balanced accuracy for class 1 (the at-risk group).

- **Recall** measures how many of the actual at-risk individuals the model correctly identifies.
 - For example, if there are 100 people who are actually at risk, and the model only identifies 20 of them correctly, the recall is very low (20%).
- **Precision** measures how many of the people predicted as at risk are actually at risk.
 - For example, if the model predicts 100 people as at risk, but only 18 of them are truly at risk, the precision is very low (18%).

The reason for this is that the model tends to predict most cases as class 0 (not at risk) because there are many more healthy people than at-risk individuals in the dataset.

As a result, the balanced accuracy, which considers the performance on both classes equally, is much lower than the overall accuracy. This shows that the model struggles to detect the minority group and needs improvements using data balancing, feature engineering, and model optimization.

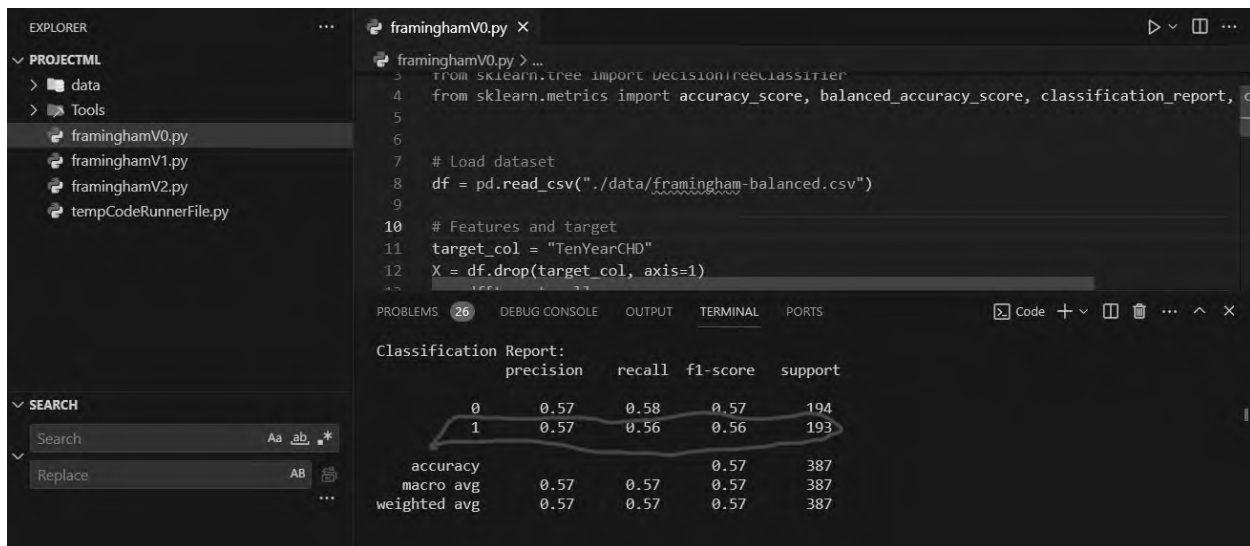
To achieve the balanced dataset, we created a Python script named split.py (can find in folder named Tools)

This script performs the following steps:

1. Splits the dataset by class (TenYearCHD = 0 and TenYearCHD = 1).
2. Downsamples the larger class to match the size of the smaller class.
3. Combines and shuffles the two classes into a balanced dataset.
4. Saves the balanced dataset as a new CSV file (framingham_balanced.csv).
5. So we using in next version framingham_balanced.csv.

This ensures that the Decision Tree model receives a perfectly balanced dataset, which improves recall and precision for the at-risk group.

So again, we run this version (framinghamV0.py) with new file



```
framinghamV0.py > ...
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score, balanced_accuracy_score, classification_report, co
5
6
7 # Load dataset
8 df = pd.read_csv("../data/framingham-balanced.csv")
9
10 # Features and target
11 target_col = "TenYearCHD"
12 X = df.drop(target_col, axis=1)
```

PROBLEMS 26 DEBUG CONSOLE OUTPUT TERMINAL PORTS

Classification Report:

	precision	recall	f1-score	support
0	0.57	0.58	0.57	194
1	0.57	0.56	0.56	193
accuracy			0.57	387
macro avg	0.57	0.57	0.57	387
weighted avg	0.57	0.57	0.57	387

We found that after balancing the dataset using split.py, the model's performance on both classes became more comparable.

We also tried techniques like SMOTE, SMOTEENN, and class_weight='balanced', but after testing, downsampling with split.py gave the best improvement in recall and precision for the at-risk class.

- **SMOTE / SMOTEENN:** generate synthetic samples for the minority class to balance the dataset, but sometimes they introduce "noise" or unrealistic values.
- **class_weight='balanced':** assigns higher weight to the minority class during training, but it does not change the actual number of samples—it just forces the model to pay more attention to the smaller class.

After testing all these approaches, we found that direct downsampling of the majority class (as in split.py) worked best. This creates a perfectly balanced dataset, improving recall and precision for the at-risk class without adding noise or complexity.

The model achieved an accuracy of 57/100, but the performance was balanced across both classes. We improved the results in the next version.

VERSION 2 (framinghamV1.py):

In this version, we improved the Decision Tree model using GridSearchCV and post-pruning.

- **GridSearchCV** is a built-in feature in scikit-learn that helps us automatically find the best hyperparameters for the tree.
- **Post-pruning** is applied after the tree is fully grown to reduce its size and prevent overfitting.

We found that post-pruning works better than pre-pruning because it allows the tree to learn fully first and then carefully remove unnecessary branches.

Using GridSearchCV makes it easier to find the optimal parameters, and post-pruning ensures the tree does not overfit, improving performance on real-world data.

Hyperparameter Tuning with GridSearch

We used GridSearchCV to find the best hyperparameters for the decision tree.

Different values for max_depth, min_samples_split, min_samples_leaf, criterion, and class_weight were tested.

The model was evaluated using balanced accuracy with 5-fold cross-validation.

The combination that gave the best score was selected for the final model.

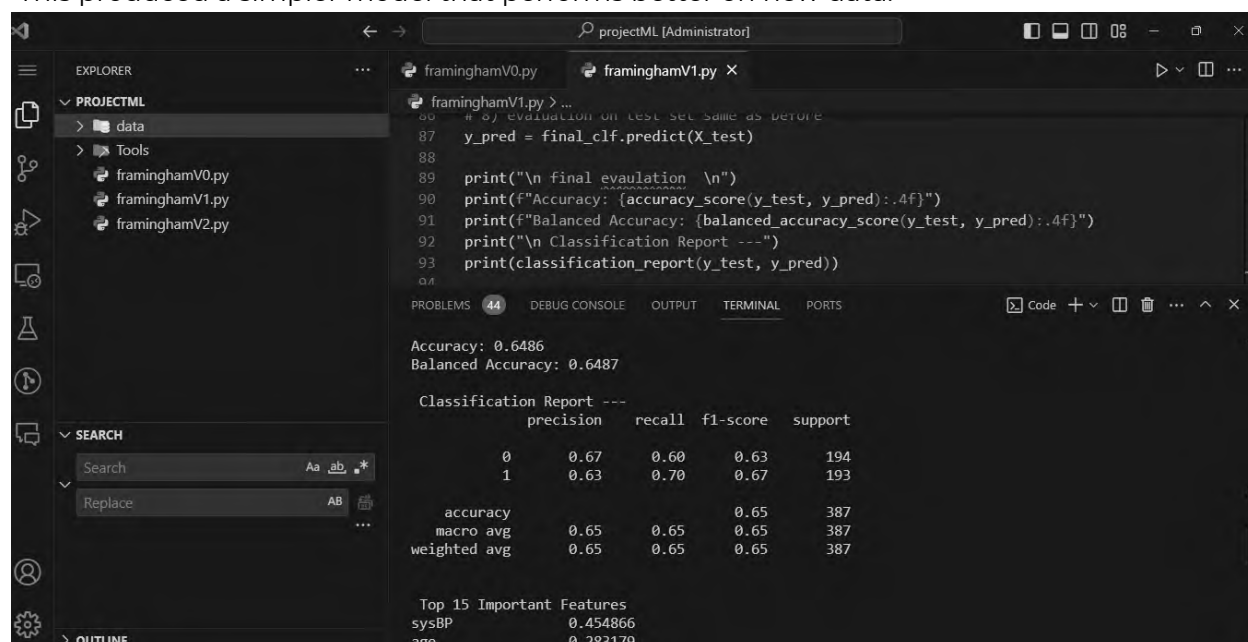
Pruning the Decision Tree

After training the tree, we applied pruning to make it simpler and reduce overfitting.

We tested different ccp_alpha values using the training data and measured balanced accuracy.

The alpha that gave the best score was chosen, and the tree was retrained with it.

This produced a simpler model that performs better on new data.



The screenshot shows the Visual Studio Code interface with a project named 'projectML'. The Explorer panel on the left shows the file structure with 'data', 'Tools', and three Python files: 'framinghamV0.py', 'framinghamV1.py', and 'framinghamV2.py'. The Search panel is also visible. The main editor shows the code in 'framinghamV1.py', which includes a function for evaluation on the test set. The output in the terminal shows the accuracy and balanced accuracy of the model, along with a classification report and the top 15 important features.

```
Accuracy: 0.6486
Balanced Accuracy: 0.6487

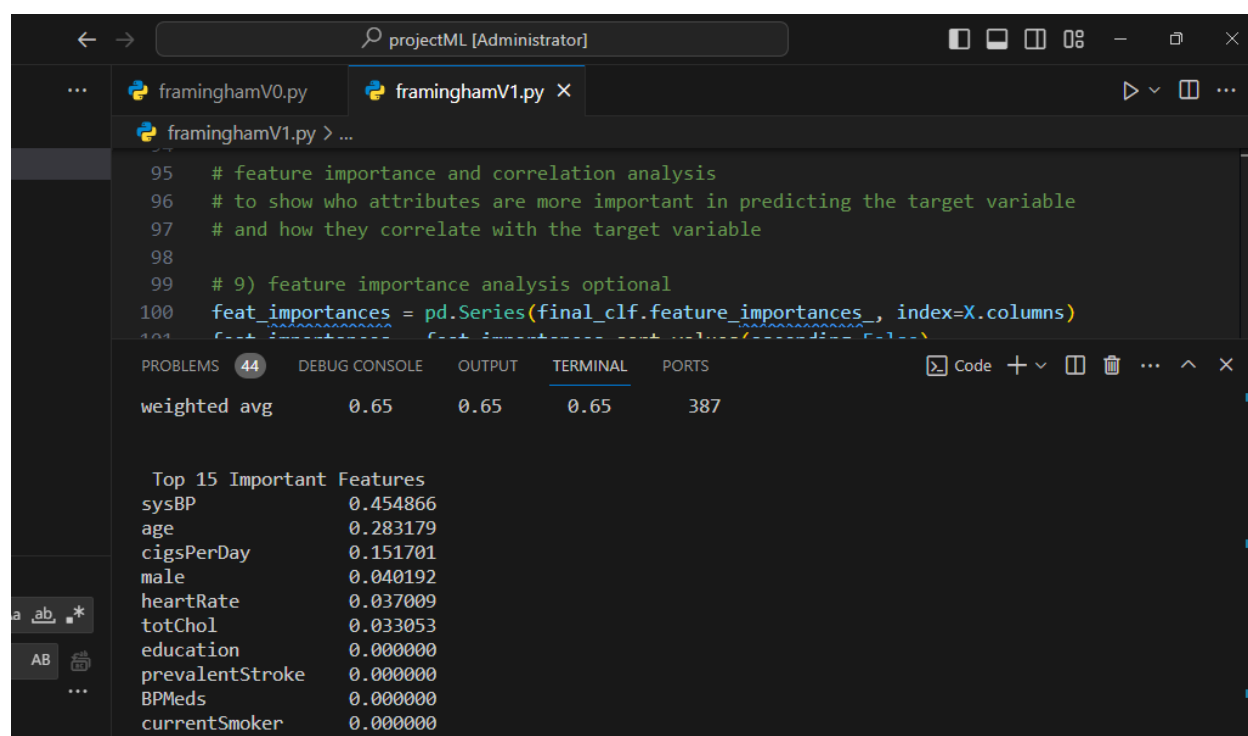
Classification Report ---
              precision    recall  f1-score   support

     0       0.67       0.60       0.63       194
     1       0.63       0.70       0.67       193

 accuracy          0.65          0.65          0.65          387
  macro avg       0.65          0.65          0.65          387
 weighted avg     0.65          0.65          0.65          387

Top 15 Important Features
sysBP      0.454866
age        0.283179
```

In this run, accuracy reached 65% thanks to better hyperparameters using GridSearch and pruning. Next, we will check how feature engineering can improve the model. Before that, we want to better understand how the decision tree works in scikit-learn by reviewing feature importance, feature influence on splits, and correlations. We wrote a simple function to print these results, and we include its output below.



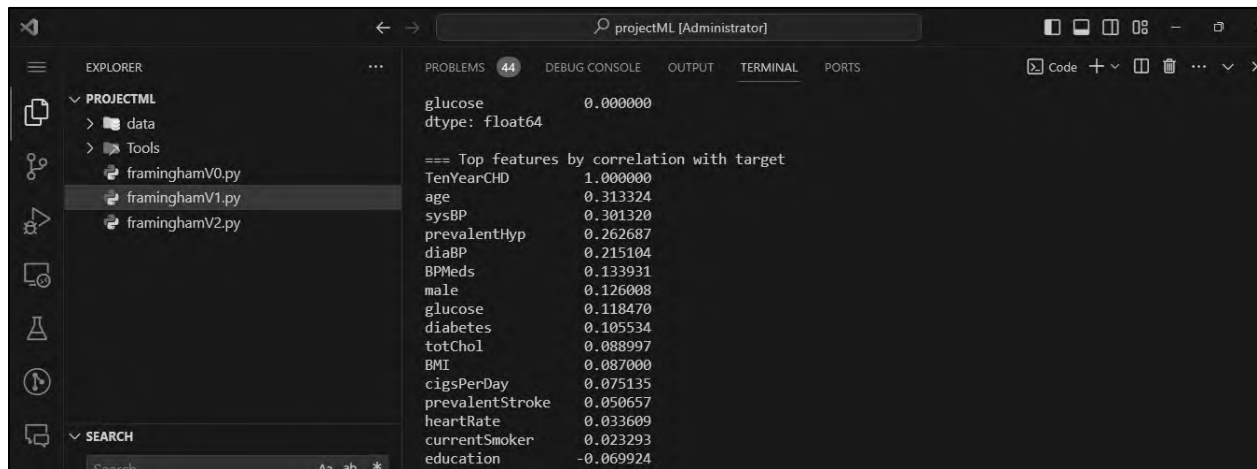
The screenshot shows the Visual Studio Code interface with the same project. The main editor shows the code in 'framinghamV1.py', which includes a function for feature importance analysis. The output in the terminal shows the weighted average of the feature importance scores, along with the top 15 important features and their scores.

```
weighted avg     0.65          0.65          0.65          387

Top 15 Important Features
sysBP      0.454866
age        0.283179
cigsPerDay 0.151701
male       0.040192
heartRate  0.037009
totChol    0.033053
education  0.000000
prevalentStroke 0.000000
BPMeds     0.000000
currentSmoker 0.000000
```

This runs show:

The feature importance values show that the model relies mainly on three key attributes: systolic blood pressure (45%), age (28%), and cigarettes per day (15%). Together, these three features represent almost 90% of the model's decision-making power, meaning the remaining features contribute very little and were not useful for the Decision Tree.

A screenshot of a Jupyter Notebook interface. The left sidebar shows a file explorer with 'PROJECTML' containing 'data', 'Tools', and three Python files: 'framinghamV0.py', 'framinghamV1.py', and 'framinghamV2.py'. The main area is split into 'PROBLEMS' and 'TERMINAL' tabs. The 'TERMINAL' tab displays the output of a script, showing 'glucose' with a value of 0.000000 and dtype 'float64', followed by a table titled '=== Top features by correlation with target'.

Feature	Correlation
TenYearCHD	1.000000
age	0.313324
sysBP	0.301320
prevalentHyp	0.262687
diaBP	0.215104
BPMeds	0.133931
male	0.126008
glucose	0.118470
diabetes	0.105534
totChol	0.088997
BMI	0.087000
cigsPerDay	0.075135
prevalentStroke	0.050657
heartRate	0.033609
currentSmoker	0.023293
education	-0.069924

Based on correlation and feature importance analysis, the key attributes selected for the model are systolic blood pressure (sysBP), age, cigarettes per day (cigsPerDay), and prevalent hypertension (prevalentHyp). These features contribute the most to predicting TenYearCHD, while other features have minimal impact.

So, we deduce who can build new feature if needed example :

- **Pulse Pressure (sysBP – diaBP)** shows the difference between systolic and diastolic blood pressure.
- **Interaction features** like age × BMI or age × smoking status reveal hidden relationships between variables.

These features help the model detect patterns and improve predictions.

Less important features, such as education or BPMeds, can be removed to make the dataset smaller and cleaner.

Version 3 – framinghamV3.py

In **Version 3** of the project, several improvements were introduced to enhance the Decision Tree model:

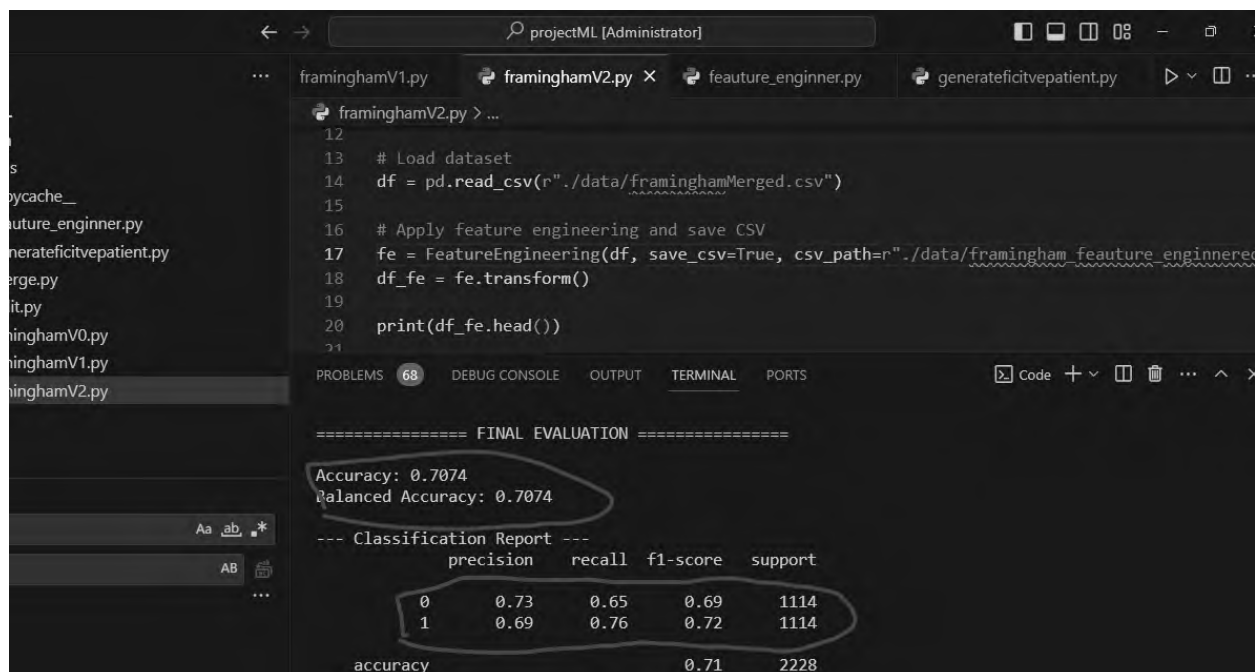
1. Merging Multiple Data Sources

- We have two data sources: framingham-balanced.csv and framingham-balancedV2.csv.
- Using the script merge.py (included in the tools folder), we merged the two files.
- The merged dataset was saved as framinghamMerged.csv, consolidating all records into a single file for further analysis.

2. Feature Engineering

- New features were created to capture relationships between attributes.
- Example: Calculating the difference between systolic and diastolic blood pressure (Pulse Pressure) to better represent cardiovascular risk.
- Eliminate noise in dataset.

After run can show :



```
framinghamV2.py > ...
12
13 # Load dataset
14 df = pd.read_csv(r"./data/framinghamMerged.csv")
15
16 # Apply feature engineering and save CSV
17 fe = FeatureEngineering(df, save_csv=True, csv_path=r"./data/framingham_feature_engineered.csv")
18 df_fe = fe.transform()
19
20 print(df_fe.head())
21
```

===== FINAL EVALUATION =====

Accuracy: 0.7074
Balanced Accuracy: 0.7074

--- Classification Report ---

	precision	recall	f1-score	support
0	0.73	0.65	0.69	1114
1	0.69	0.76	0.72	1114
accuracy			0.71	2228

Now we move to the final step: using the model for prediction with the CGTAN library. We wrote a script in the tools folder to create fictive patients (realistic data) and then evaluate the model on this data:

After completing the training of the final Decision Tree classifier, we saved the trained model to disk so it can be reused later for prediction without retraining. We used Joblib, which is the standard library for saving machine-learning models in Python.

The model is saved as a file named model.pkl in the root directory of the project.

We saved the Decision Tree model that we trained in the file framinghamV3.py, and then we placed the saved model inside a folder called Model_prediction.

After that, we retrained the model on data generated for synthetic patients, which were created using a script called GenerateFictivePatient.py, located inside the Tools folder.

Next, we wrote another script named prediction.py, which generates 10 random patients (or any number we choose, and from any category). The script then performs predictions for all generated patients.

The results are displayed by comparing:

- the model's predicted outcome
- the actual label that was assigned during data generation

It also shows both:

- the **prediction result**.
- the **prediction probability** generated by the model.

```
35 # Interleave rows (alternating 0,1,0,1,...)
36 interleaved = pd.DataFrame()
37
38 for i in range(min_count):
39     interleaved = pd.concat([interleaved, df0_bal.iloc[[i]], df1_bal.iloc[[i]]], ignore_index=True)
40
41 # Save the interleaved balanced dataset
42 output_file = "../data/framingham-balancedV2.csv"
43 interleaved.to_csv(output_file, index=False)
44
45 print(f"Balanced alternating rows file saved as: {output_file}")
46 print(interleaved.head(10))
47
```

Run prediction

===== FINAL PATIENTS (WITH COMPARISON) =====

	male	age	education	...	Pred	Prob	TenYearCHD_generated
0	1	53	1.006769	...	1	0.509091	0
1	0	45	1.021274	...	0	0.186207	0
2	0	59	4.051689	...	0	0.362445	0
3	1	54	1.010192	...	0	0.471744	0
4	0	67	1.018085	...	0	0.185279	0
5	0	72	1.002940	...	0	0.361111	0
6	0	52	0.976004	...	0	0.185279	0
7	0	65	3.996023	...	1	0.515528	1
8	0	40	3.017537	...	0	0.100000	1
9	1	59	1.037087	...	1	0.846341	1

[10 rows x 25 columns]

80/10 correct

In summary, we created 10 patients and performed prediction on them successfully.

For the output of the 10 generated users, we decided to make it dynamic, meaning both the patients and their predictions are created automatically.

As you can see from the results, out of the 10 synthetic patients, the model correctly predicted 8 of them. This corresponds to approximately the same accuracy we obtained earlier (around 70% in model of decisiontree represent in Framingham V2. py). So the new dynamic test confirms the model's performance is consistent with our initial evaluation.

Conclusion

This project demonstrates a complete machine learning workflow applied to the Framingham Heart Study dataset, starting from raw data and ending with a fully optimized and deployable Decision Tree prediction model. Across multiple versions, we progressively improved model performance by applying essential data science techniques, including:

- **Data cleaning** and handling missing values
- **Balancing the dataset** to improve detection of at-risk individuals
- **Hyperparameter tuning** using GridSearchCV
- **Decision Tree pruning** to reduce overfitting
- **Feature engineering**, including Pulse Pressure and interaction features
- **Model evaluation** using accuracy, precision, recall, and balanced accuracy
- **Synthetic data generation** using CTGAN to simulate new patients
- **Saving the final model** as a .pkl file for real-time prediction using joblib
- **Dynamic prediction testing** on randomly generated patients and evaluate with

Target

Python Libraries Used in This Project

Below is the full list of libraries used throughout the development:

Data Handling & Preprocessing

- **pandas** – for loading, cleaning, merging, and transforming datasets
- **numpy** – for numerical operations and array computations

Machine Learning (scikit-learn)

- **sklearn.tree.DecisionTreeClassifier** – the core algorithm
- **sklearn.model_selection.GridSearchCV** – hyperparameter tuning
- **sklearn.model_selection.train_test_split** – splitting data
- **sklearn.metrics** – accuracy, precision, recall, f1-score, balanced accuracy

Synthetic Data Generation

- **CTGAN (from SDV library)** – generating realistic synthetic patients

Model Saving & Loading

- **joblib** – saving the optimized Decision Tree model into .pkl format

Project Structure (Short Explanation of Each File)

projectML/

Main project folder that contains all scripts, data, tools, and the final model.

data/

Folder where original or processed datasets are stored (e.g., *framingham.csv*, balanced files, merged files).

Model_prediction/

Contains the saved machine learning models:

- **model.pkl** → the final trained Decision Tree model
- **ctgan_model.pkl** → the trained CTGAN model used to generate synthetic patients

Tools/

Contains custom helper scripts used for data processing and generating synthetic patients.

◆ **feature_enginner.py**

Script for feature engineering:

Creates new features like Pulse Pressure, interactions, or other transformations.

◆ **generatefictivepatient.py**

Generates synthetic (fake but realistic) patient data using CTGAN.

◆ **merge.py**

Merges multiple datasets (e.g., balanced datasets) into one final CSV file.

◆ **split.py**

Balances the dataset by:

- equalite of 2 classes
- reduce the larger class
- balanced both 2 class by reduce the larger class

◆ **framinghamV0.py**

Version 0 of the model.

Basic Decision Tree on raw data — no preprocessing, no balancing, no tuning.

◆ **framinghamV1.py**

Version 1 of the model.

Adds:

- GridSearchCV hyperparameter tuning
- Decision Tree pruning
- feature importance analysis

◆ **framinghamV2.py**

Version 2 of the model.

Includes improvements like:

- Balanced and merge more than dataset
- Feature engineering

◆ **generated_patients.csv**

File containing synthetic patients created by generatefictivepatient.py.

◆ **predetection.py**

Final prediction script.**It:**

- loads the saved model
 - loads/generated synthetic patients
 - predicts CHD for each patient
 - shows prediction + probability
 - compares prediction with the true synthetic label
-

Note

We provided detailed comments inside each Python file to explain the most important parts of the code and to make the project easier to understand.

Before running any script, make sure to install all the required Python libraries.